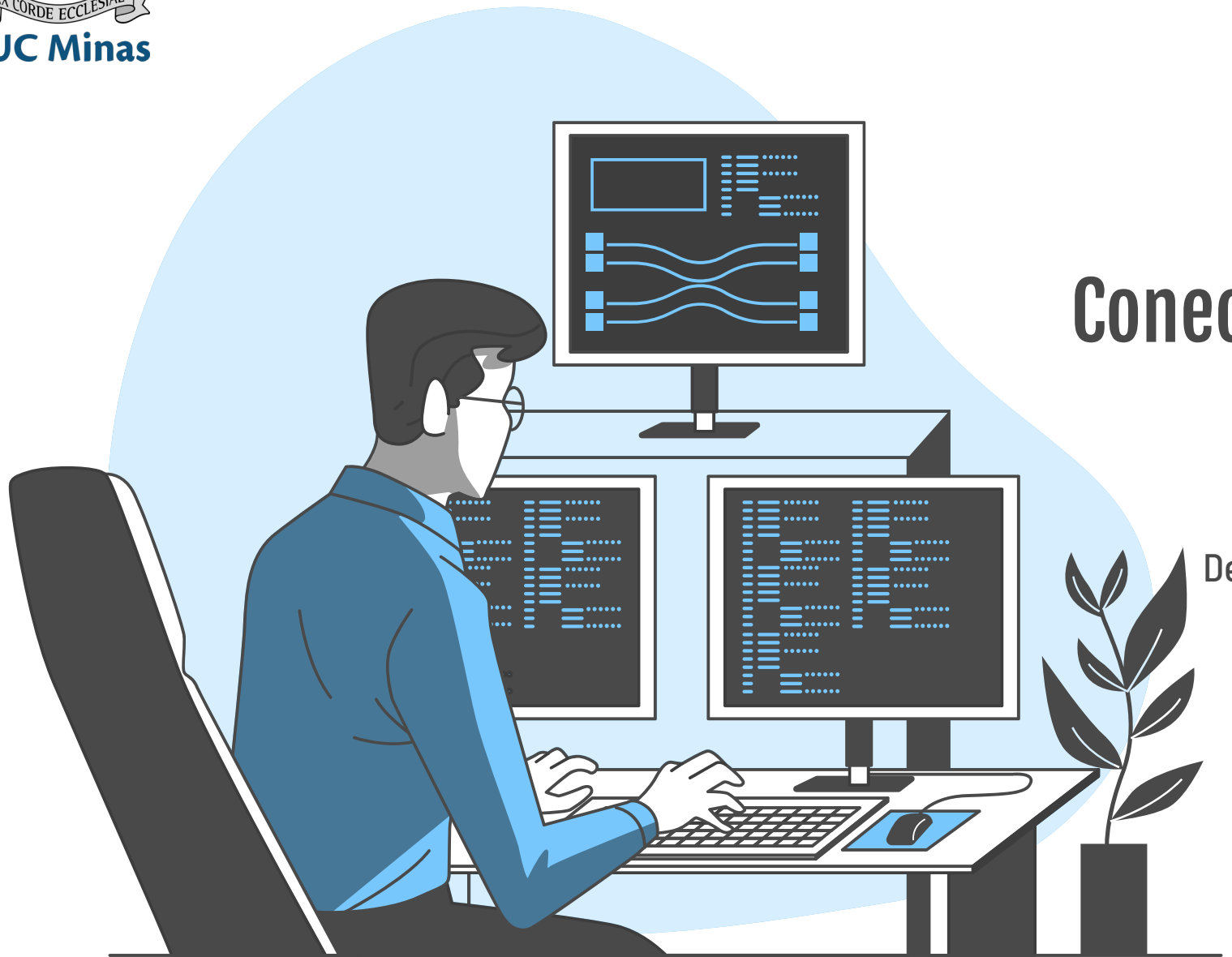
PUC Minas



Aula 04 · FullStack

Conectando o React ao Spring Boot

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Centralizar o endereço da API em `config/apiConfig.js` com `VITE_API_URL`
2. Escrever um `httpClient.js` que trata rede, status e o JSON de erro do Spring
3. Criar *services* por recurso (`alunoService.js`)
4. Implementar `GET /api/status` no back e um painel de status no front
5. Diagnosticar falhas de conexão rapidamente

Antes de começar

- Aula 03: CORS configurado

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 As três camadas do front

02 1. apiConfig.js

03 2. httpClient.js

04 3. Um service por recurso

05 4. Endpoint de status

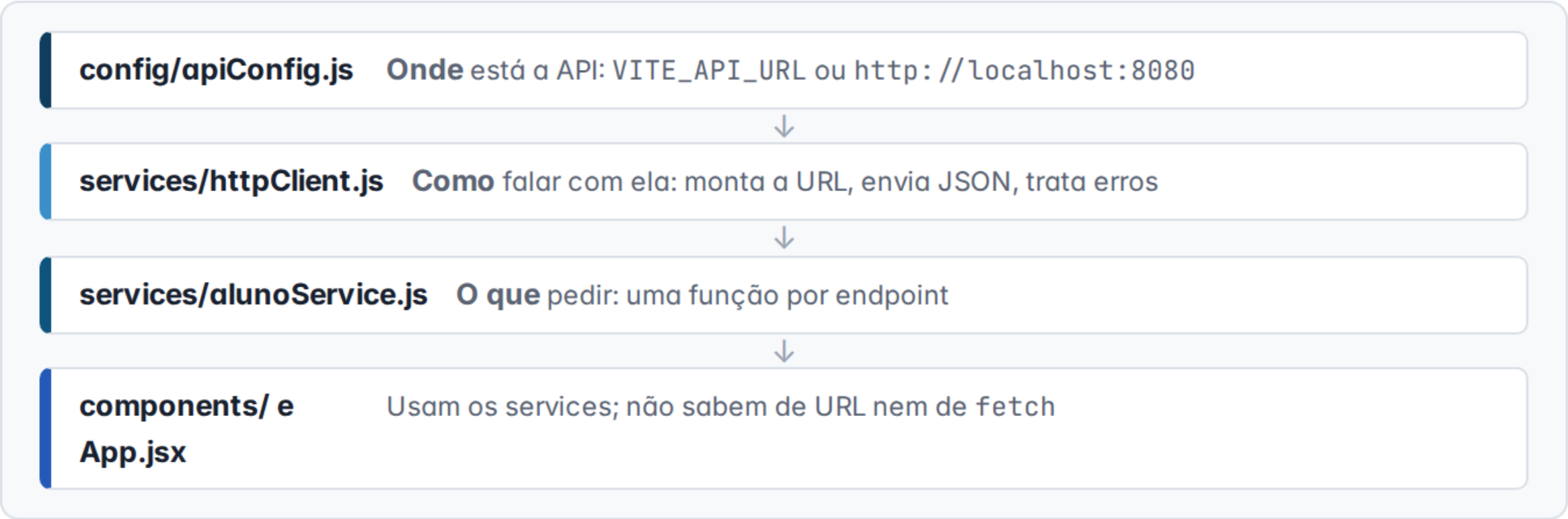
06 Diagnóstico rápido

07 Aprofundando: uma camada de comunicação madura

01

As três camadas do front

As três camadas do front



Organização do front do `Exemplo_Integracao`.

02

1. apiConfig.js

1. apiConfig.js

src/config/apiConfig.js

```
const URL_PADRAO = "http://localhost:8080";

const API_CONFIG = {
  // Remove a barra do final, se houver ("...com/" vira "...com")
  BASE_URL: (import.meta.env.VITE_API_URL || URL_PADRAO).replace(/\/+$/, ""),
};

export default API_CONFIG;
```

1. apiConfig.js

Sem `.env.local`, o front aponta para `localhost:8080`. No Vercel, a variável `VITE_API_URL` aponta para o Render. Nenhuma linha de código muda entre os ambientes.

02

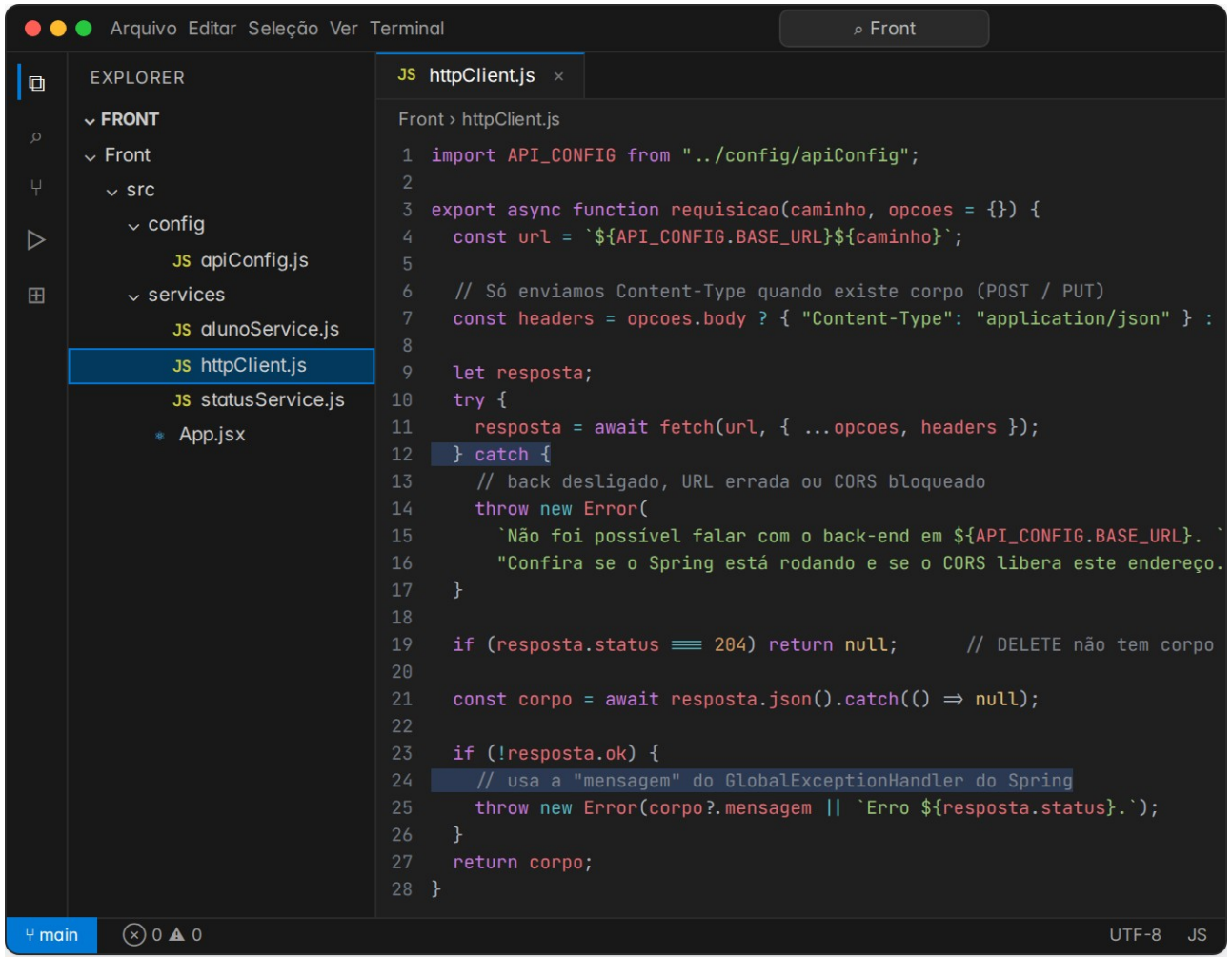
1. apiConfig.js

Aula 04 · FullStack

03

2. httpClient.js

2. httpClient.js



Uma única função para todas as chamadas. Ela distingue os dois tipos de falha:

`httpClient.js`: falha de rede vira uma mensagem de diagnóstico; erro HTTP vira a `mensagem` do Spring.

04

3. Um service por recurso

3. Um service por recurso

src/services/alunoService.js

```
import { requisicao } from "./httpClient";

const CAMINHO = "/api/alunos";

export function listarAlunos(nome = "") {
  const filtro = nome.trim() ? `?nome=${encodeURIComponent(nome.trim())}` : "";
  return requisicao(`${CAMINHO}${filtro}`);
}
export const buscarAlunoPorId = (id) => requisicao(`${CAMINHO}/${id}`);
export const criarAluno = (aluno) =>
  requisicao(CAMINHO, { method: "POST", body: JSON.stringify(aluno) });
export const atualizarAluno = (id, aluno) =>
  requisicao(`${CAMINHO}/${id}`, { method: "PUT", body: JSON.stringify(aluno) });
export const excluirAluno = (id) => requisicao(`${CAMINHO}/${id}`, { method: "DELETE" });
```

3. Um service por recurso

Observação

Repare no paralelo com o back-end: `AlunoController` tem cinco métodos, `alunoService.js` tem cinco funções com os mesmos verbos e caminhos. Quando os dois lados seguem o mesmo contrato, a integração é quase mecânica.

05

4. Endpoint de status

4. Endpoint de status

Antes de carregar dados, o front pergunta se o back e o banco estão no ar. No back:

controller/StatusController.java

```java
@RestController
@RequestMapping("/api/status")
public class StatusController {
  private final StatusService service;
  // construtor...

  @GetMapping
  public Map<String, String> status() {
    return Map.of(
      "backend", "online",
      "mongodb", service.mongoOnline()
          ? "online" : "offline");
  }
}
```

service/StatusService.java

```java
@Service
public class StatusService {
  private final MongoTemplate mongo;
  // construtor...

  public boolean mongoOnline() {
    try {
      mongo.executeCommand("{ ping: 1 }");
      return true;
    } catch (Exception e) {
      return false;
    }
  }
}
```

4. Endpoint de status

src/services/statusService.js

```
import { requisicao } from "./httpClient";

export async function verificarStatus() {
  try {
    return await requisicao("/api/status");          // { backend, mongodb }
  } catch {
    return { backend: "offline", mongodb: "desconhecido" };
  }
}
```

4. Endpoint de status

No `App.jsx` do projeto, um `useEffect` chama `verificarStatus()` ao abrir e **tenta de novo a cada 5 segundos** enquanto algo estiver fora. É útil porque o Vite sobe em 1 segundo, mas o Spring leva alguns segundos (e o Render gratuito pode levar quase um minuto para “acordar”).

05

4. Endpoint de status

Aula 04 · FullStack

4. Endpoint de status

App.jsx (trecho)

```
useEffect(() => {
  let ativo = true;
  let timer;
  const verificar = async () => {
    const novo = await verificarStatus();
    if (!ativo) return;
    setStatus(novo);
    if (novo.backend !== "online" || novo.mongodb !== "online") {
      timer = setTimeout(verificar, 5000);            // tenta de novo
    }
  };
  verificar();
  return () => { ativo = false; clearTimeout(timer); };
}, []);
```

06

Diagnóstico rápido

Diagnóstico rápido

1. Abra `http://localhost:8080/api/status` **direto no navegador**. Se não responder, o problema é o back (não está rodando, porta errada, erro no console do Spring).

2. Se responder no navegador mas o React falha, abra o **Console**: erro de CORS → aula 03.

3. Se não há erro de CORS, abra a aba **Network** e confira a URL chamada. Ela usa a `BASE_URL` certa? Tem barra dupla (`//api`)?

4. Se o status vem `4xx/5xx`, leia o corpo da resposta: o `mensagem` do Spring diz o que houve.

5. Para conferir qual URL o front está usando, mostre `API_CONFIG.BASE_URL` na tela (o painel do `Exemplo_Integracao` faz isso).

07

Aprofundando: uma camada de comunicação madura

Aprofundando: uma camada de comunicação madura

Repetir GETs quando o Render está acordando

```
export async function requisicao(caminho, opcoes = {}, tentativas = 2) {
  try {
    return await requisicaoUnica(caminho, opcoes);
  } catch (e) {
    const podeRepetir = !opcoes.method || opcoes.method === "GET";   // só idempotentes
    if (podeRepetir && tentativas > 0 && e.rede) {
      await new Promise((r) => setTimeout(r, 1500));
      return requisicao(caminho, opcoes, tentativas - 1);
    }
    throw e;
  }
}
```

Aprofundando: uma camada de comunicação madura

Timeout: não deixe o usuário esperando para sempre

```
export async function requisicaoUnica(caminho, opcoes = {}, ms = 15000) {
  const controle = new AbortController();
  const timer = setTimeout(() => controle.abort(), ms);
  try {
    return await fetch(`${API_CONFIG.BASE_URL}${caminho}`, { ...opcoes, signal: controle.signal });
  } finally {
    clearTimeout(timer);
  }
}
```

Aprofundando: uma camada de comunicação madura

Situação	O que o usuário deve ver
Back acordando (Render)	“Servidor iniciando, aguarde…” com nova tentativa automática
Back fora do ar	“Não foi possível falar com o servidor” + botão tentar de novo
Erro 400	A `mensagem` do Spring, perto do campo
Erro 404	“Registro não encontrado” e volta à lista
Erro 500	Mensagem genérica; detalhe só no log do servidor

Services com fetch x axios

fetch + httpClient próprio

- Sem dependências
- Você controla cada detalhe
- Usado nos projetos

vs

axios.create()

- `baseURL`, `timeout` e interceptadores prontos
- Erros 4xx/5xx viram exceção
- Mais uma dependência

Aprofundando: uma camada de comunicação madura

A mesma camada com axios

```
import axios from "axios";
export const api = axios.create({ baseURL: API_CONFIG.BASE_URL, timeout: 15000 });

api.interceptors.response.use(
  (r) => r.data,
  (e) => Promise.reject(new Error(e.response?.data?.mensagem ?? "Falha de comunicação")),
);

export const listarAlunos = () => api.get("/api/alunos");
```

Aprofundando: uma camada de comunicação madura

Dica

Mostre a `BASE_URL` em algum canto da tela (como o painel do `Exemplo_Integracao`) ou no console em desenvolvimento. Metade dos problemas de integração é o front apontando para o endereço errado.

08

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Qual arquivo decide onde está a API?

A httpClient.js

B config/apiConfig.js

C App.jsx

D vite.config.js

Resposta

Qual arquivo decide onde está a API?

A httpClient.js

✓ config/apiConfig.js

C App.jsx

D vite.config.js

Por quê: Ele lê VITE_API_URL ou usa localhost:8080.

Pergunta

O que o httpClient faz com uma resposta 400 do Spring?

A Ignora

B Lança erro com o campo `mensagem` do JSON

C Recarrega a página

D Tenta de novo

Quiz · questão 2 de 5

Resposta

O que o httpClient faz com uma resposta 400 do Spring?

A Ignora

✓ Lança erro com o campo `mensagem` do JSON

C Recarrega a página

D Tenta de novo

Por quê: O componente só exibe a mensagem.

Quiz · questão 3 de 5

Pergunta

Para que serve `GET /api/status`?

A Listar alunos

B Saber se o back e o banco estão no ar

C Fazer login

D Gerar token

Resposta

Para que serve `GET /api/status`?

A Listar alunos

✓ Saber se o back e o banco estão no ar

C Fazer login

D Gerar token

Por quê: O front tenta de novo até ficar online.

Pergunta

Qual o primeiro passo para diagnosticar falha de conexão?

A Reinstalar o Node

B Abrir a URL do back direto no navegador

C Apagar o banco

D Trocar de navegador

Resposta

Qual o primeiro passo para diagnosticar falha de conexão?

A Reinstalar o Node

✓ Abrir a URL do back direto no navegador

C Apagar o banco

D Trocar de navegador

Por quê: Se o back não responde sozinho, o problema não é o front.

Pergunta

Por que tirar a barra final da BASE_URL?

A Estética

B Evitar URLs com `//api`

C Exigência do CORS

D Para o HTTPS

Resposta

Por que tirar a barra final da BASE_URL?

A Estética

✓ Evitar URLs com `//api`

C Exigência do CORS

D Para o HTTPS

Por quê: `.replace(/\/+$/, "")`.

Fechamento

Resumo da aula

- ✓ `apiConfig.js` decide **onde** (VITE_API_URL ou localhost:8080).
- ✓ `httpClient.js` decide **como**: monta URL, envia JSON, trata falha de rede e lê `mensagem` dos erros.
- ✓ Um service por recurso, espelhando os endpoints do controller.
- ✓ `GET /api/status` + tentativa periódica deixam o front resiliente ao back que demora a subir.
- ✓ Diagnóstico: back no navegador → Console (CORS) → Network (URL) → corpo do erro.

Exercícios

1. Crie `apiConfig`, `httpClient` e `livroService` para a sua API de livros.

2. Implemente `GET /api/status` no seu back com o *ping* do banco (Mongo ou um `SELECT 1` no Supabase).

3. Mostre no front um aviso “Servidor iniciando…” enquanto o status não estiver online.

4. Desligue o back com o front aberto e confirme que a mensagem exibida ajuda o usuário.

Para ir além

- Projeto `FullStack/Exemplo_Integracao/Front/src/services`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 04?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web